# CSCC43

2024-08-30

# 1 Relational Model

**Theorem 1A.** techniques to scale DBMS

1. memory cache: optimise processing speed
2. index keys
3. parallel processing
4. distributed systems: separate storage

**Definition 1A.** a *(relation) schema* consists of

- name of the relation $R$
- set of *attributes* $A_1, \cdots, A_n$

$$R(A_1, \cdots, A_n)$$

a set of schemas is a *database schema*

**Definition 1B.** an *instance* of a schema $R(A, B, C)$ consists of a set of *tuples*

| $A$ | $B$ | $C$ |
|---|---|---|
| $a_1$ | $b_1$ | $c_1$ |
| $a_2$ | $b_2$ | $c_2$ |

**Definition 1C.** a *key* of a relation $R(A, B, C)$ is a subset of attributes that ...

$$R(\underline{A}, B, C)$$

# 2 Entity Relationship Model

## 2.1 ER Model

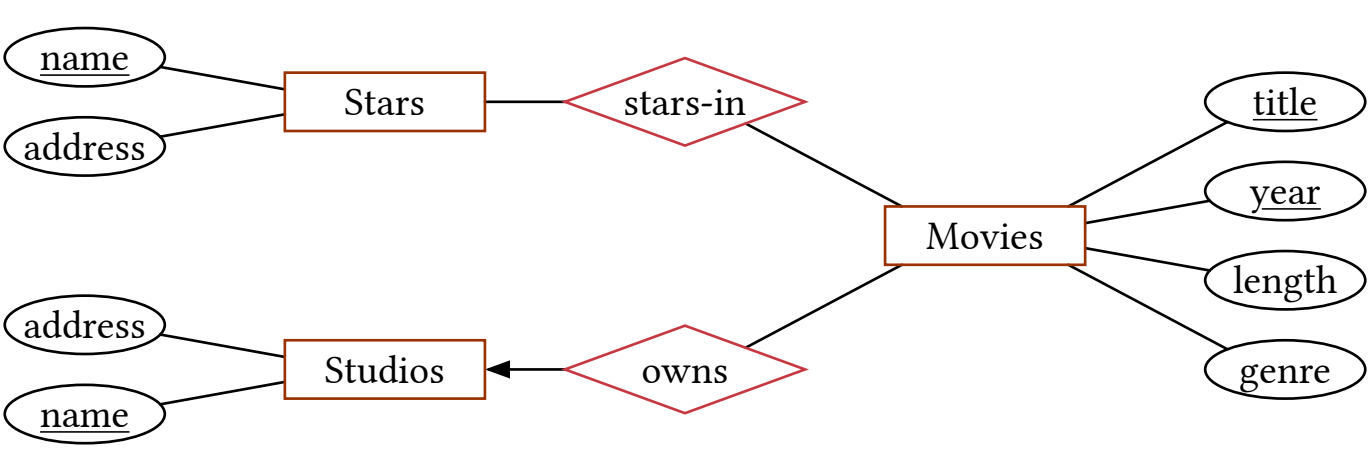
**Definition 21A.** an *ER model* consists of

- *entity sets*: objects
- *attributes*: properties of an entity
- *relationships*: connections among esets

**Definition 21B.** an *ER diagram* is a graph representing an ER model

- esets are rectangles, attributes are ovals, relationships are diamonds

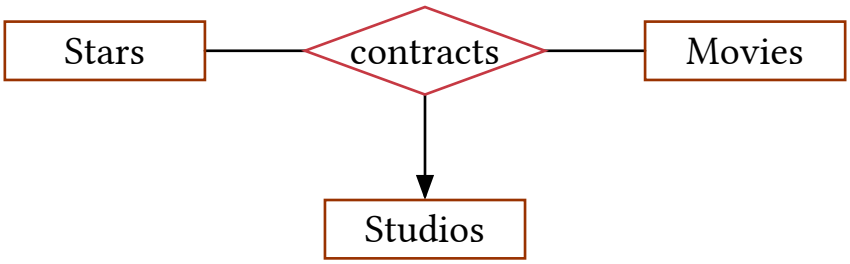
**Example 21A.** ER diagram with keys



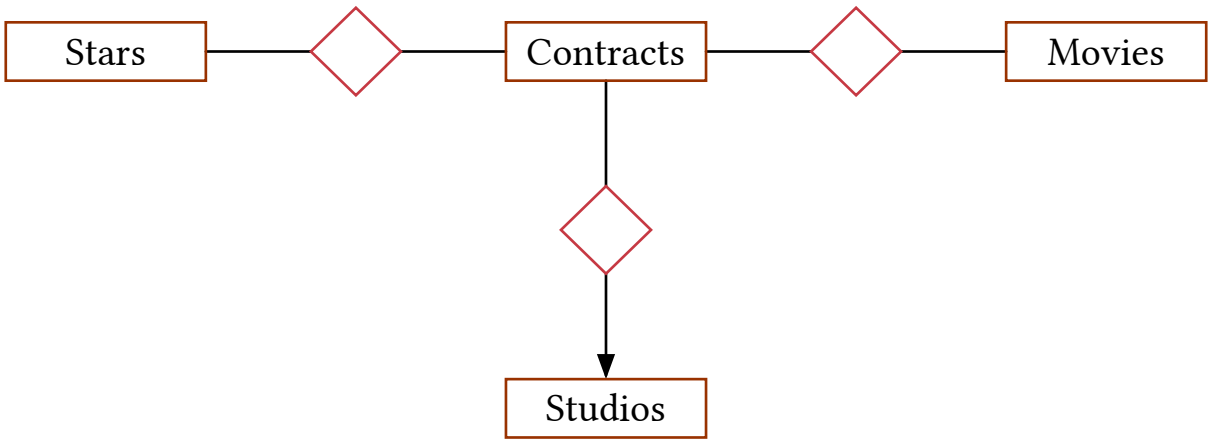

**Definition 21C.** multiplicity of binary ER relationships

| notation | multiplicity |
|---|---|
| — | many-many (relation) |
| $\longrightarrow$ | many-one (function) |
| $\longleftrightarrow$ | one-one (injection) |

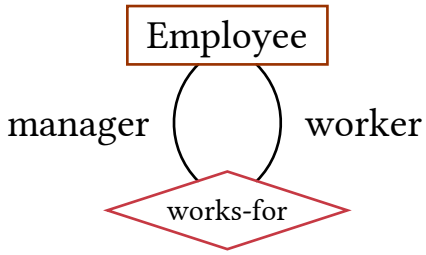
**Theorem 21A.** multiway relationship



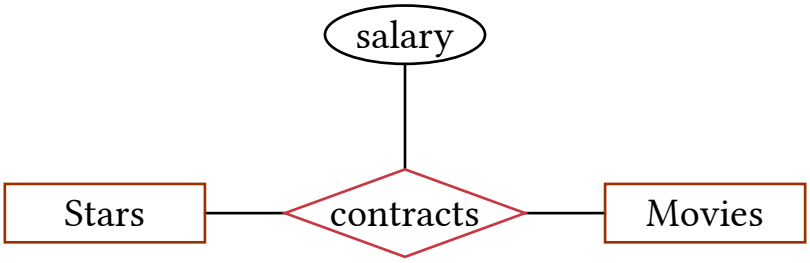

is shorthand for



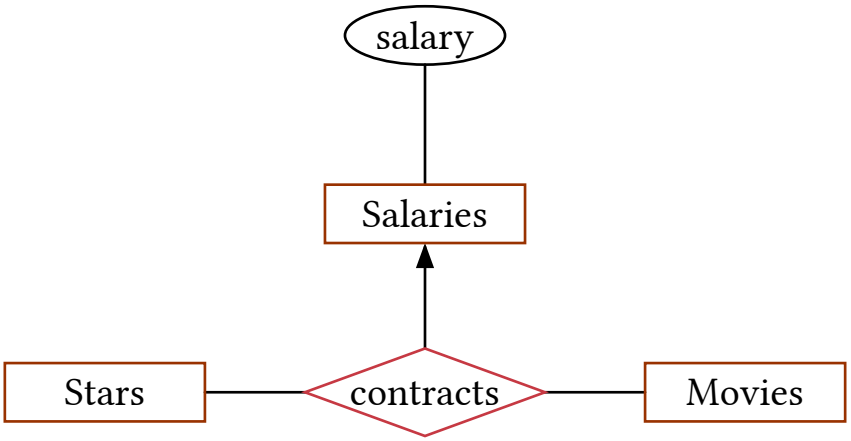

**Theorem 21B.** relationship with roles




**Theorem 21C.** relationship with attributes

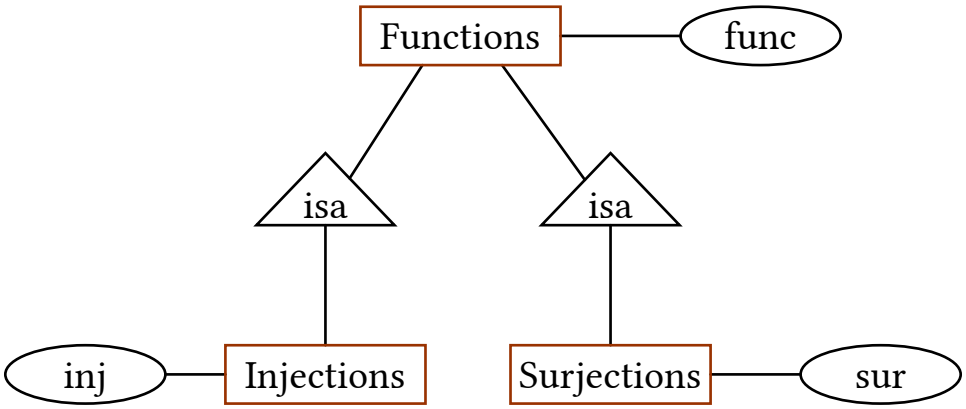



is short hand for




**N.B.** many to one

**Example 21B.** subclass (can't think of better example)




## 2.2 Weak Entity Sets

**Definition 22A.** weak entity sets

**N.B.** `Species.name` is only unique within a `Genus`
- `belongs` is the **supporting** relationship of `Species`
- `Genus` is the **supporting** entity set

## 2.3 ER Diagram to Relation Schema

**Theorem 23A.** entity to schema

- entity attrs $\longrightarrow$ schema attrs
- entity keys $\longrightarrow$ schema keys

**Theorem 23B.** relationship to schema

- connecting entity keys $\longrightarrow$ schema attrs
- relationship attrs $\longrightarrow$ schema attrs (never key)
- (many-many) schema attrs $\longrightarrow$ schema keys
- (many-one) "many" entity keys $\longrightarrow$ schema keys
- (one-one) pick one "one" entity keys $\longrightarrow$ schema keys
- roles $\longrightarrow$ rename attrs

**Theorem 23C.** weak entity to schema

- same as entity
- supporting keys $\longrightarrow$ schema keys
- supporting relationship attribute $\longrightarrow$ schema attrs
- supporting relationship don't need to be converted

## 2.4 Subclass to Relation Schema

**Theorem 24A.** ER style vs OO style vs None

- *ER style*: one relation for each eset, include root keys
- *OO style*: one relation for each subtree, relations partition
- *None*: one relation with all attrs, use $\perp$

**Example 24A.** see Example 21B

- ER style

- OO style

$$\text{Functions}(\underline{\text{func}})$$
$$\text{Injections}(\underline{\text{func}}, \text{inj})$$
$$\text{Surjections}(\underline{\text{func}}, \text{surj})$$

- None style

$$\text{Funcs}(\underline{\text{func}})$$
$$\text{FuncsI}(\underline{\text{func}}, \text{inj})$$
$$\text{FuncsS}(\underline{\text{func}}, \text{surj})$$
$$\text{FuncsIS}(\underline{\text{func}}, \text{inj}, \text{surj})$$

$$\text{Functions}(\underline{\text{func}}, \text{inj}, \text{surj})$$

**Theorem 24B.** comparasion

queries
- *None*: one relation, so cheap
- queries with only attrs from root is cheap in *ER*, but expensive in *OO*
- queries with attrs from different esets is cheap in *OO*, but expensive in *ER*

relation amount for root with $n$ childrens
- *ER*: $O(n)$ relations
- *OO*: $O(2^n)$ relations
- *None*: one relation

space
- *ER*: only repeats keys
- *OO*: zero redundency
- *None*: long tables

# 3 Relational Algebra

## 3.1 Operations

**Definition 31A.** set operations

for relations $R, S$ with the same attributes
- $R \cup S$ union
- $R \cap S$ intersection
- $R - S$ difference
- $R \times S$ cartisian product
  - any common attribute $A \in R \cap S$ is renamed to $R.A$ and $S.A$

**Definition 31B.** projection and selection

for relation $R(A_1, \cdots, A_n)$

- **projection**: for $L = A_1 \cdots A_k \subseteq R$
$$\pi_L R = \{(a_1, \cdots, a_k) : (a_1, \cdots, a_k, \cdots, a_n) \in R\}$$
- **selection**: for condition $C(r)$
$$\sigma_C R = \{r \in R : C(r)\}$$

**Definition 31C.** join

- **natural join**: agree on common attributes
$$R \bowtie S \equiv \pi_{R \cup S} \sigma_C (R \times S) \qquad C := \forall A \in R \cap S \quad R.A = S.A$$
- **theta join**: select product on some condition $C$
$$R \underset{C}{\bowtie} S \equiv \sigma_C (R \times S)$$
- **semi join**: project one side (same $C$ from natural join)
$$R \ltimes C \equiv \pi_R \sigma_C (R \times S)$$
- **outer join**: natural join or pad null, $R \bowtie_o S$
  - outer *left* join $R \bowtie_{oL} S$ and outer *right* join $R \bowtie_{oR} S$

**Definition 31D.** more operators

- **extended projection**:
  - $\pi_{A \to B}$ for rename attr $A$ to attr $B$
  - $\pi_{e \to B}$ for evaluate expression $e$ as attr $B$
- **rename**: for $R(A_1, \cdots, A_n)$ and $S(B_1, \cdots, B_n)$
$$\rho_S R \equiv \pi_L R \qquad L = A_1 \to B_1, \cdots, A_n \to B_n$$
- **dedup**: $\delta R$
- **sort**: $\tau_L R$ where $L \subseteq R$ is list of tuples
- **group**: $\gamma_L R$ where $L$ is list of attributes or aggregation operators
- **division**:
$$R/S = \{r \in \pi_{R-S} R : \forall s \in S \quad (r, s) \in R\}$$

## 3.2 Bag/Multiset Semantics in SQL

**Definition 32A.** a bag is $(X, m)$ where $X$ is set and $m : X \to \mathbb{Z}^+$ is multiplicity

**Definition 32B.** semantics

- **union**: union sets, sum of multiplicity
- **intersection**: intersect sets, minimum of multiplicity
- **difference**: difference sets, substract multiplicity

**N.B.** $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ don't hold for bags

# 4 Design Theory

## 4.1 Functional Dependencies

**Definition 41A.** functional dependency

$$A_1 A_2 \cdots A_n \to B_1 B_2 \cdots B_m$$

**Definition 41B.** key of relations

for relation $R(A_1, \cdots, A_n)$
- $K \subseteq R$ is a **superkey** of $R$ iff $K \to R$
- $K \subseteq R$ is a **key** iff is minimal superkey

## 4.2 Reason about Functional Dependencies

**Definition 42A.** follows & equivalences

for sets of FD's $\mathcal{F}_1$ and $\mathcal{F}_2$
- $\mathcal{F}_1$ **follows from** $\mathcal{F}_2$ iff any relation instance that satisfies $\mathcal{F}_2$ also satisfies $\mathcal{F}_1$
- $\mathcal{F}_1$ and $\mathcal{F}_2$ are **equivalent** iff one follows from another

**Definition 42B.** Armstrong's axioms

sound and complete inference rules for FD's
- *reflexivity*
$$Y \subseteq X \vdash X \to Y$$
- *augmentation*
$$X \to Y \vdash XZ \to YZ$$
- *transivity*
$$X \to Y, Y \to Z \vdash X \to Z$$

**Theorem 42A.** rules (follows from Armstrongs)

$$X \to AB \vdash X \to A, X \to B$$
$$X \to A, X \to B \vdash X \to AB$$
$$X \to A, AB \to C \vdash XB \to C$$

**Definition 42C.** closure of attributes

for attributes $X$ and FDs $\mathcal{F}$
- $X^+$ is the closure of $X$ under $\mathcal{F}$

**N.B.** $K \subseteq R$ is superkey of relation $R$ iff $K^+ = R$

**Definition 42D.** basis of FD's

- for FD's $\mathcal{F}$, any set of FD's equivalent to $\mathcal{F}$ is a **basis** of $\mathcal{F}$

a **minimal** basis is $\mathcal{B}$ such that
- all FD's in $\mathcal{B}$ are in from $X \to A$ (i.e. singleton on right)
- no strict subset of $\mathcal{B}$ is basis
- for any $X \to A \in \mathcal{B}$, remove an attribute from $X$ makes NOT basis

**Algorithm 42A.** basis to minimal basis

given basis $\mathcal{F}$
1. **strengthen**: for any $XA \to B \in \mathcal{F}$
   if $\mathcal{F} \vdash X \to B$, then replace $XA \to B$ with $X \to B$ in $\mathcal{F}$
2. **eliminate**: for any $F = X \to A \in \mathcal{F}$
   if $\mathcal{F} - \{F\} \vdash X \to A$, remove $X - A$ from $\mathcal{F}$
3. repeat until $\mathcal{F}$ no change

**Algorithm 42B.** FD projection

to project $(R, \mathcal{F})$ onto $\pi_L R$
1. let $X^+_{\mathcal{F}}$ be closure of $X$ for any $X \subseteq \pi_L R$
2. let $\mathcal{T}$ be all nontrivial $X \to A$ such that $A \in X^+$
3. let $\mathcal{B}$ be minimal basis computed from $\mathcal{T}$

## 4.3 Anomalies and BCNF

**Definition 43A.** anomalies

anomalies happen when we cram too much into a single relation
- **redundancy**: information repeats (especially when FD exists)
- **update anomaly**: when certain attribute repeats, update only one tuple creates inconsistency
- **delete anomaly**: when multiple esets combined into one relation, delete an instance in one eset can cause delete an instance in other eset

**Definition 43B.** decomposition

to avoid anomalies, decompose $R$ into $R_1$ and $R_2$ such that
- $R = R_1 \cup R_2$ (as attributes)
- $R_i = \pi_{R_i} R$

**Definition 43C.** BCNF

relation $(R, \mathcal{F})$ is in **BCNF** iff for any nontrivial $X \to A \in \mathcal{F}$
- $X$ is a superkey

BCNF means on anomalies

**Algorithm 43A.** BCNF decomopsition

- lossless, no dependency perserve

given relation $(R, \mathcal{F})$ that violates BCNF on $X \to Y$

1. let $R_1 = X^+$ and $R_2 = X \cup (R - X^+)$
2. project $\mathcal{F}$ onto $(R_1, \mathcal{F}_1)$, project onto $(R_2, \mathcal{F}_2)$
3. do recursively

## 4.4 Lossless Join and Dependency Perservation

**Definition 44A.** lossless join

subrelations $R_1, \cdots, R_n$ is **lossless** decomposition of $R$ iff

$$R = \pi_{R_1} R \bowtie \cdots \bowtie \pi_{R_n} R$$

**Theorem 44A.** diagram chase

- setup *tableau*, each $R_i$ is a row (with arbitrary components on other attributes)
- apply FD's of $R$ to deduce restrictions
- lossless iff has at least one unsubscripted row

**Example 44A.** chase

decompose $\{A, B, C, D\}$ into $\{A, D\}, \{A, C\}, \{B, C, D\}$

- setup tableau

| $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|
| $a$ | $b_1$ | $c_1$ | $d$ |
| $a$ | $b_2$ | $c$ | $d_2$ |
| $a_3$ | $b$ | $c$ | $d$ |

let $\mathcal{F} = \{A \to B, B \to C, CD \to A\}$

- by $A \to B$

| $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|
| $a$ | $b_1$ | $c_1$ | $d$ |
| $a$ | $b_1$ | $c$ | $d_2$ |
| $a_3$ | $b$ | $c$ | $d$ |

- by $B \to C$

| $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|
| $a$ | $b_1$ | $c$ | $d$ |
| $a$ | $b_1$ | $c$ | $d_2$ |
| $a_3$ | $b$ | $c$ | $d$ |

- by $CD \to A$

| $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|
| $a$ | $b_1$ | $c$ | $d$ |
| $a$ | $b_1$ | $c$ | $d_2$ |
| $a$ | $b$ | $c$ | $d$ |

thus lossless □

**Theorem 44B.** BCNF don't perserve dependency

for $(R, \mathcal{F})$ BCNF decomposed into $(R_1, \mathcal{F}_1)$ and $(R_2, \mathcal{F}_2)$
- for some instance of $R_1$ and $R_2$, then $R_1 \bowtie R_2$ might violate some $F \in \mathcal{F}$

## 4.5 Third Normal Form

**Definition 45A.** 3NF

relation $(R, \mathcal{F})$ is in **3NF** iff for any nontrivial $X \to Y \in \mathcal{F}$
- $X$ is a superkey, or
- $\forall A \in Y - X \quad \exists K \quad A \in K$

**Algorithm 45A.** 3NF decomposition

- lossless join, perserve dependency

given relation $(R, \mathcal{F})$ that violates 3NF
- let $\mathcal{B}$ be a minimal basis of $\mathcal{F}$
- for any $X \to A \in \mathcal{B}$, let $XA$ be a subrelation
- if none of $XA$ is a superkey, add another subrelation $K$ that is key

## 4.6 Multivalue Dependencies and 4NF

**Definition 46A.** multivalue dependency

$$X \twoheadrightarrow Y$$

iff for any $(x_1, y_1, z_1)$ and $(x_2, y_2, z_2) \in R$ where $z_i \in Z = R - XY$

$$x_1 = x_2 \implies (x_1, y_2, z_1), (x_2, y_1, z_2) \in R$$

**Theorem 46A.** MVD reasoning

- $Y \subseteq X \vdash X \twoheadrightarrow Y$
- $X \twoheadrightarrow Y, Y \twoheadrightarrow Z \vdash X \twoheadrightarrow Z$
- $X \to Y \vdash X \twoheadrightarrow Y$
- $X \twoheadrightarrow Y \vdash X \twoheadrightarrow R - Y$

- $X \twoheadrightarrow YZ \nvdash X \twoheadrightarrow Y \wedge X \twoheadrightarrow Z$

**Definition 46B.** 4NF

relation $R$ is in **4NF** iff for any nontrivial $X \twoheadrightarrow Y$
- $X$ is a superkey

**Algorithm 46A.** 4NF decomposition

given relation $R$ that violates 4NF on some $X \twoheadrightarrow Y$
- let $R_1 = X \cup Y$ and $R_2 = X \cup (R - Y)$
- project FDs and MVDs onto $R_1$ and $R_2$

# 5 Structured Query Langage

## 5.1 Three Valued Logic

**Definition 51A.** basically
- `True` is 1, `False` is 0, $\perp$ is $1/2$
- `AND` is min, `OR` is max, `NOT` is $\lambda x.1 - x$
- any operation with $\perp$ gives $\perp$

## 5.2 SQL Queries

let `M` be Movie, `E` be MovieExec, `S` be MovieStars

**Example 52A.** $\sigma$ and $\pi$

```
SELECT * FROM Movies WHERE studio='Disney' AND year=1990;
SELECT title AS name, length/60.0 AS duration, 'hours' AS unit FROM Movies;
```

**Example 52B.** (patterns) _ matches a char, % match a string

```
SELECT title FROM M WHERE title LIKE 'Star ____';
```

**Example 52C.** (multirelation) producers of Star Wars

```
SELECT name FROM M, E
  WHERE title = 'Star Wars' AND M.producer = E.producer;
```

**Example 52D.** (multirelation) stars living in the same address

```
SELECT Star1.name, Star2.name FROM S Star1, S Star2
  WHERE Star1.address = Star2.address AND Star1.name > Star2.name;
```

**Example 52E.** (set semantics) intersection, difference, union

```
(SELECT name FROM S WHERE gender = 'F')
  INTERSECT -- notice same schema
```

```
  (SELECT name from E WHERE net-worth > 1);
(SELECT name FROM S) EXCEPT (SELECT name FROM E);
(SELECT name FROM S) UNION (SELECT name FROM E);
```

**Example 52F.** (subquery) movies with same length as "Movie"

```
SELECT * FROM Movies
  WHERE length IN (SELECT length FROM Movies WHERE title='Movie');
```

**Example 52G.** (subquery) stars that have unique birth date

```
SELECT name FROM S S1
  WHERE NOT EXISTS
    (SELECT * FROM S S2 WHERE S1.bday = S2.bday AND S1.name ≠ S2.name);
```

**Example 52H.** (quantifier) longest movie

```
SELECT * FROM M WHERE length ≥ ALL(SELECT length FROM M);
```

**Example 52I.** (join) producers of "Movie"

```
SELECT name FROM M JOIN E ON M.producer = E.producer WHERE title = 'Movie';
```

**Example 52J.** bag union

```
(SELECT * FROM M) UNION ALL (SELECT * FROM E);
```

**Example 52K.** set select

```
SELECT DISTINCT name FROM S;
```

**Example 52L.** group by **N.B.** group occurs after selection

```
SELECT year, AVG(length) FROM M GROUP BY year;
```

**Example 52M.** select group

```
SELECT name, SUM(length) FROM M NATURAL JOIN E
  GROUP BY name HAVING MIN(year) < 1930;
```

## 5.3 Database Modifications

**Definition 53A.** data types

- `BOOLEAN INT REAL`
- `CHAR(n)` fixed length string — faster (no need to check length)
- `VARCHAR(n)` variable length, up to $n$ — less storage
- `DATE TIME`

**Example 53A.**

```
CREATE TABLE M(
  s VARCHAR(32),
```

```sql
  a INT DEFAULT 0,
  b INT NOT NULL,
  c INT UNQIUE,
  PRIMARY KEY(s),
  UNIQUE(a,b));
INSERT INTO M(s,a,b,c) VALUES ('a',0,0,0), ('b',1,1,1); -- insert rows
INSERT INTO M(s,a,b,c) SELECT CONCAT(s, "0"), a, b, c * 2 FROM M; -- insert result
DELETE FROM M WHERE a > 1; -- delete rows
UPDATE M SET c = 0 WHERE b > 1; -- update rows
ALTER TABLE M ADD d REAL DEFAULT 0.0; -- add new attribute `d`
ALTER TABLE M DROP a; -- drop attribute
DROP TABLE M; -- drop table
```

# 6 Constraints, Triggers, Views, Indices

## 6.1 Constraints

- data query language
- data manipulation language
- data definition language
- **data control language**

see psql

### 6.1.1 Foreign Keys

```sql
CREATE TABLE ta (
  a REFERENCES tb (b)
)
```

- Any value in `ta.a` must also exist in `tb.b`
- `tb.b` must be the primary key

Violation:

```sql
FOREIGN KEY (a) REFERENCES tb (b)
  ON DELETE CASCADE
  ON UPDATE SET NULL
```

- Detault: reject
- Cascade: ripple changes
- Set null: change the referred tuples components to NULL

### 6.1.2 Attribue-based Checks

```sql
CHECK (condition)
```

- Check only occurs during insert or update, not delete

### 6.1.3 SQL Assertions

```sql
CREATE ASSERTION <name> CHECK (<condition>)
```

- Check when table changes

## 6.2 Triggers

see psql

- Event: changes
- Condition: where-clause
- Action: SQL statements

```
CREATE TRIGGER <name> { BEFORE | AFTER } { <event> [ OR ... ] }
  ON <table>
  REFERENCING
    OLD TABLE AS <name>
    NEW TABLE AS <name>
  FOR EACH ROW
  WHEN ( <condition> )
```

todo check psql doc

## 6.3 Views

https://www.postgresql.org/docs/current/sql-createview.html

```
CREATE VIEW <name> AS <query>
```

## 6.4 Indices

# 7 Entity-Relation Model

# 8 Relational Algebra

# 9 Structured Query Language

6.1 6.2 6.3 6.4 6.5

# 10 Constraints and Triggers

7.1 7.2 7.3 7.4 7.5

### 10.1.1 Foreign Keys (Referential Integrity Constraints)

```sql
— refers to PK of t1
CREATE TABLE t2 (
    a integer REFERENCES t1
);
— specify, t1.a must have UNIQUE
CREATE TABLE t2 (
    a integer REFERENCES t1 (a)
);
— refer group of columns, (t1.a, t1.b) must have UNIQUE
```

```sql
CREATE TABLE t2 (
    a integer, b integer,
    FOREIGN KEY (a, b) REFERENCES t1 (a, b)
);
```

**Two ways to violate constraint**

- Insert/update `t2` row so it refers to nonexistent in `t1`
  - reject
- Delete/update `t1` row
  - default: reject
  - `REFERENCES t1 ON DELETE CASCADE`: propagate changes
  - `REFERENCES t1 ON UPDATE SET NULL`: set to `NULL`
  - `REFERENCES t1 ON UPDATE SET DEFAULT`: set to (default)

### 10.1.2 Attribute-based Checks

```sql
CREATE TABLE orders (
  amount numeric CHECK (amount > 0)
);
CREATE TABLE people (
  gender CHAR(1), name TEXT,
  -- female implies Ms.
  CHECK (gender = 'F' OR name NOT LIKE 'Ms. %')
)
```

- Check only on insert/update

### 10.1.3 SQL Assertions (Global Constraints)

```sql
CREATE ASSERTION name CHECK
  (NOT EXISTS (SELECT * FROM t1, t2 WHERE <undesirable_condition>))
```

### 10.1.4 Triggers (Event-Condition-Action)

```sql
-- only allow increase
CREATE TRIGGER name
AFTER UPDATE OF a ON t1 -- event
REFERENCING
  OLD ROW AS orow,
  NEW ROW AS nrow,
FOR EACH ROW -- (trigger called once for each row updated)
WHEN (orow.a > nrow.a) -- condition
  UPDATE t1 SET a = orow.a; -- action
```

# 11 Views and Indices

8.1 8.3

```sql
CREATE VIEW name AS { query }
```

**Example 11A.** (Optimisation)

```
CREATE VIEW v1 AS SELECT a, b FROM t1 WHERE c = 0;
SELECT a FROM v1 WHERE b = 0;
```

the query $\pi_a \sigma_{b=0}$ v1 expands to

$$\pi_a \sigma_{b=0} \pi_{a,b} \sigma_{c=0} \text{ t1}$$

which can be optimised to

$$\pi_a \sigma_{b=0,c=0} \text{ t1}$$

### 11.1.1 Indices

TODO 2024-06-24

# 12 Transactions

**Definition 12A.** Structure of DBMS

- **Query optimisation/execution**: find optimised plan, and execute
- **Relational operators**: implements relational algebra
- **File access methods**: access data on disk
- **Buffer management**: manages in-memory buffer (high performance)
- **Storage management**: controls disk activity (for persistency)

The last three are stateful, so need CC&R

**Definition 12B.** Transaction manager: controls execution of txns

6.6

## 12.1 Concurrency Control

18.1 18.2 18.3

Need concurrency because

- Disk is bottleneck of performance
- CPU has multiple cores
- Improves application response time

**Definition 121A.** ACID properties of transaction execution

- Atomicity: all actions in a txn suceed or fail together
- Consistency: if DB starts consistent, each txn is consistent, then ends consistent
- Isolation: execution of one txn is isolated from the other
- Durability: if a txn commits, the effect persist

**Theorem 121A.** (Atomicity) If txn abort, DBMS should undo the changes using loggging

**Theorem 121B.** (Consistency) Consistency is expressed as a set of **integrety constraints** (`CREATE TABLE` or `CREATE ASSERTION`)

- Txns that violates ICs should be aborted

**Theorem 121C.** (Isolation) DBMS interleaves actions of various transations, one does not step onto another

- Txn's behavior should not be impacted by other txns

**Theorem 121D.** (Durability) If system crashes before disk changes, DBMS should redo the commited txns, using logging

### 12.1.1 Schedule Anomalies

**Definition 1211A.**

- *serial schedule* is schedule without interleave txns
- two schedules are said to be *equivalent* iff have the same effect
- a schedule is *serialisable* iff equivalent to some serial schedule

**Theorem 1211A.** If each txn perserves consistency, then a serial schedule also perserves consistency (thus so does serialisable schedules)

**Example 1211A.** (Nonserialisable transactions) $a_i$ for abort, $c_i$ for commit

| | | |
|---|---|---|
| *WR conflict* | $w_1(A); r_2(A); a_1$ | Reading uncommited data |
| *RW conflict* | $r_1(A); w_2(A); r_1(A), c_1, c_2$ | Unrepeatable reads |
| *WW* conflict | $w_1(A); w_2(A); w_1(B); c_1, c_2$ | Overriding uncommited data |

- In RW conflict, the second read's behavior is undetermined
- In WW conflict, the table will have updated $A$ from $t_2$, and updated $B$ from $t_1$

To avoid anomalies, we can use locking

### 12.1.2 Two-Phase Locking (2PL) Protocol

**Definition 1212A.** (2PL) The **lock manager** implement 2PL:

- *Growing phase:* txn must acquire
  - *shared (S)* lock before reading
  - *mutex (X)* lock before writing
- *Shrinking phase:* release locks after the last (R/W) action of the txn

**Strick 2PL**: release lock after txn completes

## 12.2 Recovery

17.1 17.2 17.3 17.4

**Definition 122A.** The **recovery manager** brings DB to consistent state after crash.

- Undo commited actions
- Redo commited actions

To do so, the recovery manager uses *log records*

**Definition 122B.** If txn writes an object, the recover manager logs

- UNDO: before image of the object
- REDO: after image of the object

If txn commits/abort: record log indicating this action

### 12.2.1 Write Ahead Logging (WAL) Protocol

**Definition 1221A.** (WAL)

- Before making page dirty, record logs to permanent storage
- All log records must be written to disk (including commit record) BEFORE the txn commits

Now we cover an algorithm to perform the recovery

### 12.2.2 ARIES Recovery

**Algorithm 1222A.** (ARIES)

1. *Analysis:* scan log forward (from most recent)
   - Identify txn that were active
   - Identify dirty pages in buffer pool (at time of crash)
2. *Redo:* redo upates to dirty pages from buffer pool, make sure logged updates are updated in disk
3. *Undo:* undo uncommited transactions, read logs backwards

## 12.3 Transcations in PostgreSQL

**Definition 123A.** 4 level of isolation

1. *Serliasable:*
   - all txns occur in completely isolated
2. *Repeatable read:*
   - Cannot read after uncommited write
   - Cannot write when other txn reads (until other commits/aborts)

3. Read commited
   - Cannot read after uncommited write
   - Want to write → wait for write lock
4. Read uncommited
   - Allows read after uncommited write (not in Postgres)

**Definition 123B.** A **phantom read** (query-level) happens when the same read txn reexecutes, but returns different results, due to some recently commited txn

**Definition 123C.** A **nonrepeatable read** (datapoint-level) happens when txn rereads data without write, but the data has been modified by another txn's commited write.

**Definition 123D.** A **dirty read** occurs when txn reads data from a row that has been modified by an uncommited write from another txn

## 12.4 Deadlocks

**Definition 124A.** Two ways to deal with deadlocks
1. prevent deadlocks
2. detect and break deadlocks

### 12.4.1 Prevent Deadlocks

Priority based approach: assign priorities based on timestamp (older = higher)

**Example 1241A.** Two policies are possible, suppose $T_2$ wants a lock that $T_1$ holds
- (Wait-die) if $T_2$ has higher priority, $T_2$ waits, otherwise $T_2$ aborts
- (Wound-wait) if $T_2$ has higher prority, $T_1$ aborts, otherwise $T_2$ waits

Make sure a restarted txn gets its original timestamp, otherwise starvation

### 12.4.2 Detect Deadlocks

**Definition 1242A.** (Wait-for graph) bruh you did thread scheduling id c69

**Example 1242A.**

$$r_1(A); w_1(B); r_2(B); w_2(C); r_3(C); w_3(D); a_1();$$

Wait for graph:

$$\begin{tikzcd}
T_1 & T_2 \arrow[l, "B"'] \\
T_3 \arrow[ur, "C"] &
\end{tikzcd}$$

Therfore, need to abort $T_2, T_3$

**Example 1242B.**

$$r_2(A); r_3(A); r_1(A); w_1(B); r_3(B); w_2(C); r_3(C); a_1()$$

Wait-for graph:

$$\begin{tikzcd}
T_1 & T_2 \\
T_3 \arrow[u, "B"] \arrow[ur, "C"] &
\end{tikzcd}$$

Therefore, need to abort $T_3$

**Example 1242C.**

$$r_1(A); r_2(B); w_1(C); r_3(D); r_4(D); w_3(B); w_2(C); w_4(A); w_1(D);$$

wait-for graph

$$\begin{tikzcd}
T_1 \arrow[dr, "D"] & T_2 \arrow[l, "C"'] \\
T_4 \arrow[u, "A"] & T_3 \arrow[u, "B"']
\end{tikzcd}$$

Therefore, cycle happens

# 13 Advanced Topics

Denormalise for performance (save joins)

## 13.1 On-Line Analytic Processing

10.6

**Definition 131A.** (Measure/dependent attributes) Attributes that measures some value, can be aggregated.
- e.g. price of a sale

**Definition 131B.** (Dimension attributes) Dimensions on which measure attributes can be viewed
- e.g. product of a sale

**Definition 131C.** Common architecture
- collect data to local store branch databases
- local store DBs copied to central warehouse periodically, where OLAP is performed

### 13.1.1 Star Schema

**Definition 1311A.** A schema for multidimentional data, consists of
- *facts table:* large collection of facts, periodically updated (e.g. sales)
- *dimension table:* small, static information

(looks like a star, the fact table is the center)

**Example 1311A.** Bar chain

Fact table:

$$\text{Sales}(\underline{\text{bar}}, \underline{\text{beverage}}, \underline{\text{time}}, \text{price})$$

Dimension tables:

$$\text{Bars}(\underline{\text{name}}, \underline{\text{address}})$$

$$\text{Beverage}(\underline{\text{name}}, \text{manifacturer})$$

**Definition 1311B.** (Attributes)
- *Dimension attribute:* key of a dimension table
- *Dependent attribute:* other attributes in facts

**Definition 1311C.** Approaches to build warehouses
- *ROLAP:* tune a relational DBMS to support star schemas
- *MOLAP:* multidimensional OLAP, use a specialised DBMS, e.g. data cube

### 13.1.2 Star Schema Operations

**Definition 1312A.**
- Roll up: aggregate along some dimensions
  e.g. aggregate into total sales by each store

- Drill down: break an aggretate into constituents
  e.g. given the store with highest sales, break down into sales by product

### 13.1.3 Data Cubes

10.7

**Definition 1313A.** multidimensional generalisation of aggregates
- 3 dimension attributes → 3D cube
- each cell represents an aggregation

**Example 1313A.**

| *item_name* | *color* | *clothes_size* | *quantity* |
|---|---|---|---|
| skirt | dark | small | 2 |
| skirt | dark | medium | 5 |
| skirt | dark | large | 1 |
| skirt | pastel | small | 11 |
| skirt | pastel | medium | 9 |
| skirt | pastel | large | 15 |
| skirt | white | small | 2 |
| skirt | white | medium | 5 |
| skirt | white | large | 3 |
| dress | dark | small | 2 |
| dress | dark | medium | 6 |
| dress | dark | large | 12 |
| dress | pastel | small | 4 |
| dress | pastel | medium | 3 |
| dress | pastel | large | 3 |
| dress | white | small | 2 |

Figure 4: Table

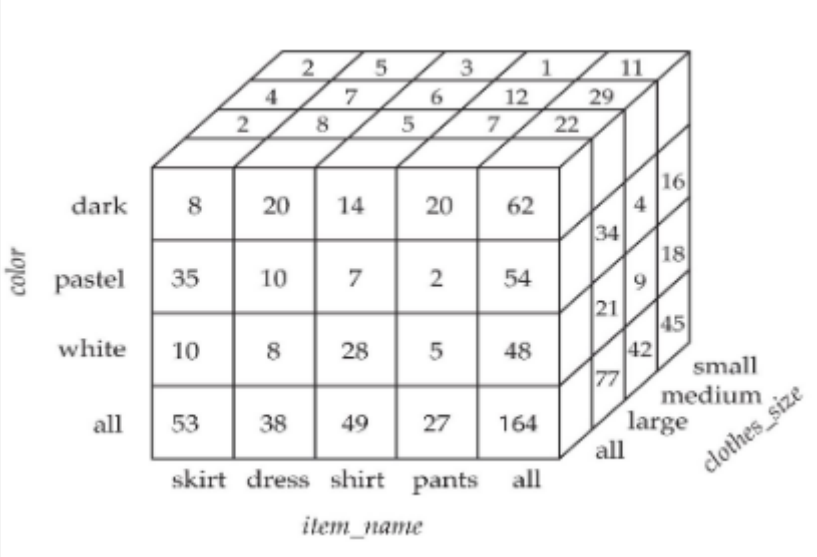



Figure 5: Cube

*clothes_size* **all**

| *item_name* | *color* dark | pastel | white | total |
|---|---|---|---|---|
| skirt | 8 | 35 | 10 | 53 |
| dress | 20 | 10 | 5 | 35 |
| shirt | 14 | 7 | 28 | 49 |
| pants | 20 | 2 | 5 | 27 |
| total | 62 | 54 | 48 | 164 |

Figure 6: Cross-tabulation/pivot-table (basically 2D cube)

*clothes_size:* **all**

| *category* | *item_name* | *color* dark | pastel | white | total | |
|---|---|---|---|---|---|---|
| womenswear | skirt | 8 | 8 | 10 | 53 | |
| | dress | 20 | 20 | 5 | 35 | |
| | subtotal | 28 | 28 | 15 | | 88 |
| menswear | pants | 14 | 14 | 28 | 49 | |
| | shirt | 20 | 20 | 5 | 27 | |
| | subtotal | 34 | 34 | 33 | | 76 |
| total | | 62 | 62 | 48 | | 164 |

Figure 7: Drill down a crosstab

To convert from crosstab to relation, use `ALL` value

| item_name | color | clothes_size | quantity |
|---|---|---|---|
| a | b | ALL | c |

SQL uses `NULL` instead of `ALL`

### 13.1.4 The CUBE Operation

**Example 1314A.**

```
SELECT name, color, size, SUM(number)
FROM sales
GROUP BY CUBE(name, color, size);
```

- `GROUP BY CUBE(a, b, c)` computes union of 8 different groupings (all subsets)
- if attribute not present, uses `NULL`
  e.g. the grouping {} has key `(NULL, NULL, NULL, s)` where `s` is agg over all rows

**Example 1314B.** If `a,b,c,d` each has 2 values, then the number of rows `CUBE(a,b,c,d)` gives is

$$\sum_{i=1}^{4} \binom{4}{n} 2^n$$

## 13.2 Data Mining

**Definition 132A.** **Market-basket** data is sets of "items" purchased together as a customer leaves a store

**Definition 132B.** Summary of basket data is **frequent itemsets**, set of items that often appear together in baskets

**Example 132A.** If people often buy burger and ketchup togther, McD can run a sale on burger and raise price of ketchup

### 13.2.1 Frequent Pairs

**Example 1321A.** To find frequent pairs, consider

$$\text{Baskets}(\text{basket}, \text{item})$$

with a **support threshold** s, minimum frequency of pair to become interested

Then

```
SELECT b1.item, b2.item
FROM Baskets b1, Baskets b2
WHERE b1.basket = b2.basket AND b1.item < b2.item — different items in same basket
GROUP BY b1.item, b2.item — group for each pair (that appears at least once)
HAVING COUNT(*) ⩾ s;
```

But the join from above is huge

**Example 1321B.** apriori optimisation

- need to have frequent items first, before consider frequent pairs

Solution: matview

```
CREATE MATERIALIZED VIEW baskets_v(basket, item) AS
  SELECT * FROM baskets
  WHERE item IN (
    SELECT item FROM baskets
    GROUP BY item HAVING COUNT(*) ⩾ s);
```

Advantage

- refreshing the view is fast, no join
- `baskets_v` has at most as many rows as `baskets`
- running time of join shrinks quadratically (in number of rows involved)

# 14 Query Execution

**Definition 14A.** Query processor: turns DML commands into DB operations

SQL

- → Query Compilation
- (Query plan) → Query Execution
- (Data) → Database files

Query compilation can also read metadata from database file

**Definition 14B.**

- Query compilation
  1. parse query into parse tree
  2. construct optimised logical plan (relational operators)
  3. generate physical execution plan (exact operations)

**Definition 14C.** Disk oriented DBMS

- (fast) Execution engine: CPU
- (fast) Buffer pool: fixed pages
- (slow) Database file: disk

## 14.1 Query Processing Models

Given plan, how to process

### 14.1.1 Iterator model

**Definition 1411A.**

- every query plan operator has a `next()`
- on invoke each `next()`, returns either a tuple or `EOF`
- the operator implements a loop that calls `next()` on its children
- each operator also implements `open()` and `close()` (constructor/destructors)

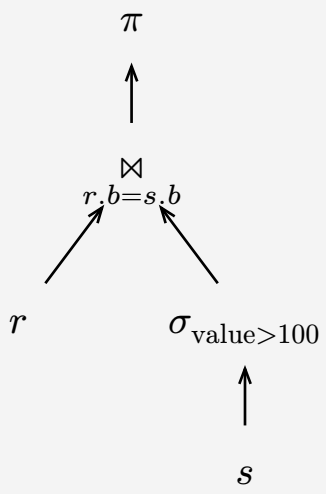
Used by most DBMS

- many operators (join, count, groupby, orderby) must **block** until all their children emit all tuples

**Example 1411A.** Iterator model

```
SELECT r.a, s.c
FROM r JOIN s ON r.b = s.b
WHERE s.value > 100
```

1. generate plan:




2. implement `next()`

```
def π_next():
  for t in child.next():
    emit(proj(t))

def ⋈_next():
  for t1 in left.next():
    buildHashTable(t1)
    for t2 in right.next():
      if probe(t2):
        emit(join(t1, t2))

def σ_next():
  [emit(t) for t in child.next() if pred(t)]

def r_next():
  [emit(t) for t in R]
```

### 14.1.2 Materialisation Model

**Definition 1412A.**

- each operator processes input all at once, and emits all at once (every operator is a materialisation point)
- DBMS can push down hints (e.g. LIMIT) to optimise
- output can be
  - a materialised row, whole tuples (NSM nary storage model)
  - subsets of columns (DSM decomposition storage model)

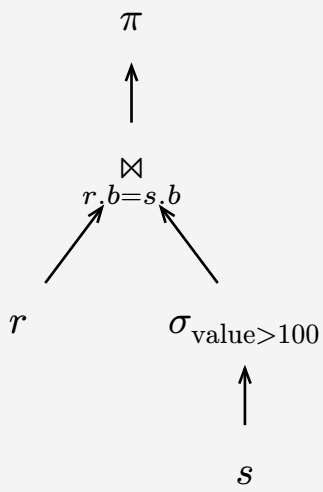
better for OLTP (txn processing workflow)

- lower execution overhead
- fewer function calls

not good for OLAP with large intermediate results

- too much memory

**Example 1412A.**




```
def π_mat():
  out = [proj(t) for t in child.output()]
  return out
```

### 14.1.3 Vectorisation Model

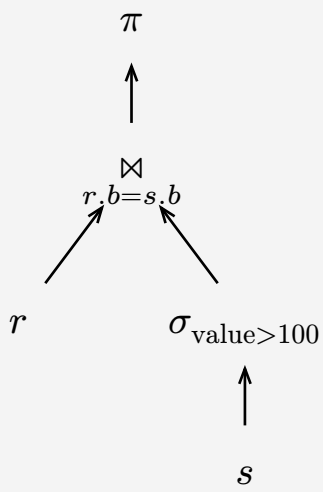
**Definition 1413A.** balance between itertor and materialisation

- each operator implements `next()`, but emits a batch of tuples

good for OLAP

- allows operator to be vectorised instructions

**Example 1413A.**




```
def π_vec():
  out = []
  for t in child.next():
    out.append(proj(t))
    if len(out) > n:
      emit(out); out = []
```

## 14.2 Plan Processing Directions

**Definition 142A.**

Top-down
- start with root, pull data up from children
- tuple passed with function calls

Bottom-up
- start execution with leaf, push data to parents
- can optimise execution plan at runtime

## 14.3 Access Methods

**Definition 143A.** (how to read tuples from table, not in relational algebra)

Sequential Scan
- scan each page from buffer pool or load from database file
- iterate over each tuple, check if wanted (e.g. $\sigma$)
- internally, DBMS maintains an internal cursor that tracks the last page
- slow, but has optimisations (e.g. prefetch pages)

Index Scan
- uses an index

Multi-index Scan
1. compute sets of record IDs with each index
2. combine these sets based on query's predicate (union/intersect bitmaps)
3. retrieve records, apply the rest of predicates

**Example 143A.**

```sql
SELECT * FROM students
WHERE age < 30 AND dept = 'cs' AND country = 'CA'
```

suppose have index on `age` and `dept`
1. 99 ppl with `age < 30` but only 2 in CS → choose `dept` index
2. 99 ppl in CS but only 2 with `age < 30` → choose `age` index

multi index scan
1. get IDs with `age < 30` using index
2. (parallelisable) get IDs with `dept = 'cs'` using index
3. intersection, check `country = 'CA'`

## 14.4 Data Modication Language

**Definition 144A.**
- INSERT/UPDATE/DELETE are responsible for

1. update target index
2. constraint check (immediately after or deferred until commit)

- UPDATE/DELETE
  1. child operators passed IDs for target tuples
  2. must keep track of seen tuples
- INSERT
  1. Choice 1, materisalise tuples inside operator, wait for multiple inserts, reduces write overhead
  2. Choice 2: inserts whatever returned by child operators, minimises latency of each insert operator

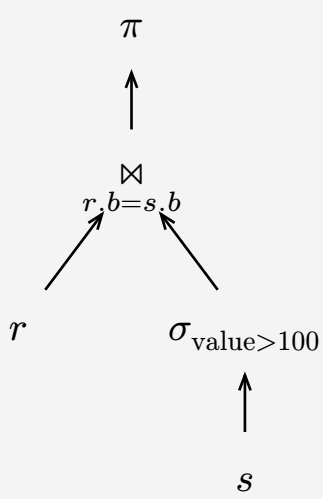
**Theorem 144A.** (Halloween problem) UPDATE need to track seen tuples, because update changes physical location, which causes scan operator to visit tuple multiple times

## 14.5 Expression Evaluation

**Definition 145A.** interpreting WHERE clauses is slow, JIT is better

## 14.6 Query Optimisation

**Example 146A.** Physical plan




- $r$: sequential scan
- $s$: B-tree index, pipline to $\bowtie$
- $\bowtie$: index nested loop, pipline to $\pi$
- $\pi$: simple projection

**Definition 146A.** Query optimiser

1. finds candidate equivalent trees (NP-hard)

2. search through a subset of all plans
3. for each plan, finds physical plan, estimates cost
4. choose best physical plan

Techniques

1. heuristics/rules: rewrite query to remove (guessed) inefficiecie (e.g. select first)
2. cost based search: estimates

**Definition 146B.** Heuristics

1. *predicate pushdown:* apply $\sigma$ first
2. *replace cartesian product:* convert $\sigma . \times$ to $\bowtie$
3. *projection pushdown:* apply $\pi$ first

**Definition 146C.** Cost estimation

1. single relation query
   - pick best access method (sequential/index)
   - decide predicate ordering
2. multi relation query (e.g. join, star schema)
   - bottom-up: starts with empty physical plan, incrementally add
   - top-down: starts with logical plan, gradually turns to physical plan
3. estimate physical costs: e.g. predict CPU cycles, RAM consumption
4. estimate logical costs: e.g. output size of operator

# 15 NoSQL

**Definition 15A.** problem with scaling

- scaling up: expensive
- scaling out (horizontal): add more smaller/cheaper servers: better choice

**Definition 15B.** scaling out approaches

1. primary/secondary
   - one primary nodes: handles all update workloads
   - multiple secondary nodes: handles read
   - issue: read latency, 2nds have to wait for 1st to write
   - issue: large dataset causes problems, 1st need to duplicate data to 2nds
2. sharding: partitions database
   - every node handles read/write
   - issue: not transparent, application needs to be aware of partitions (e.g. minimise data movement between partitions)
   - issue: integrety checking across partitions can be bottleneck
   - issue: join across partitions

3. others ways to scale out
   - multiple primary replication: but need to maintain consistent
   - no UPDATE/DELETE, only INSERT
   - no JOIN, saves query time (needs to denormalise data)

## 15.1 Not Only SQL

**Definition 151A.** features
- non relational, no tables, no schema
- scaled out: data replicated to multiple nodes (thus fault-tolerant)
- cheap/easy to implement (open-source)
- write performance
- fast key/value access

disadvantags
- no relation features (e.g. join, groupby, orderby)
- no referential integrity
- no SQL, needs programming
- CAP

examples
- Google BigTable
- Amazon Dynamo

**Theorem 151A.** (CAP) No distributed system can guarantee all of
- Consistency: all copies have same value
- Availability: reads/writes always succeed (always available to accept requests)
- Partition-tolerance: should be consistent and available when network fails

(you can have two properties)

**Example 151A.**
- traditional DBMS: prefers C over A over P (e.g. bank)
- web-applications these days: prefers A over C

## 15.2 Consistency

**Definition 152A.**

Strong consistency: ACID

Weak consistency: BASE (basically available soft-state eventual consistency)
- doesnt have to be consistent at any time
- needs to be eventually consistent

**Example 152A.**

1. Row $x$ replicated on nodes $m$ and $n$
2. client $A$ write $x$ to $n$
3. then, client $B$ reads $x$ from $m$
4. $\therefore$ $B$ *maybe* sees the write from $A$

**Definition 152B.** Eventual consistency
- when no updates occur for a long period of time, *eventually* update propagates through system, making consistent
- suits well with cloud computing, but hard to achieve ACID

## 15.3 Categories

**Definition 153A.**
- Key-value store (e.g. DynamoDB)
- Document-based (e.g. MongoDB)
- Column-based (e.g. BigTable)
- Graph-based (e.g. neo4j)

In common: no schema, provides flexible data types

**Definition 153B.** key-value stores (Dynamo)
- focuses on scaling to huge amount of data
- designed to handle massive load
- model: global collection of kv pairs (hashmap)
- issue: can't model complext data structure

features
- `get(key)` `put(key, value)` `delete(key)` `execute(key, op, param)`

**Definition 153C.** document stores
- models more complex objects
- model: collection of documents (e.g. JSON, XML)

**Definition 153D.** column stores (BigTable)
- organise data in columns
- similar to RDMBS, but handle semistructured, no schema required
- mode: collection of column families
  - (key, value), value is set of related columns

**Definition 153E.** graph database
- focuses on interconnectivity
- scales to complex data